

Advanced Multimodal and Agentic RAG

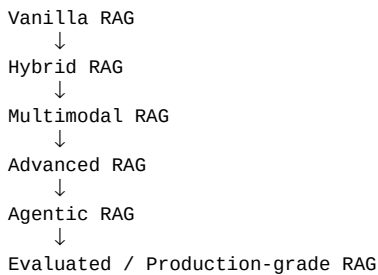
Product Requirements Document (PRD)

Project: Advanced Retrieval-Augmented Generation based Question and Answering System

1. Product Overview

Product Type: AI-powered document intelligence and question-answering platform.

Product Goal: Build a modular, scalable and intelligent RAG system that can ingest PDFs, images, CSVs and other formats; understand textual and visual information; retrieve relevant evidence using multiple retrieval strategies; generate grounded answers with an LLM; provide citations; and evaluate the quality of the complete RAG pipeline.



2. Problem Statement

- Traditional document QA systems primarily handle text.
- Scanned documents require OCR and visual understanding.
- Charts, diagrams and screenshots are difficult for text-only RAG.
- Vector-only search can miss exact keywords, names and numbers.
- Retrieved chunks may contain irrelevant information.
- LLMs can hallucinate or generate unsupported claims.
- Answers may lack reliable source citations.
- Different document versions can contain conflicting information.
- Structured data such as CSVs often requires exact computation rather than semantic retrieval.
- Many RAG systems lack rigorous benchmarking and evaluation.

3. Objectives

- Build a modular multi-format document ingestion pipeline.
- Support text, tables, images, charts, diagrams and structured data.
- Implement intelligent semantic and structure-aware chunking.
- Generate configurable embeddings and store knowledge efficiently.
- Implement hybrid vector + BM25 retrieval.
- Add reranking, query rewriting and contextual compression.
- Support multi-document and multi-hop reasoning.
- Provide citation-based, evidence-grounded answers.

- Detect insufficient and conflicting evidence.
- Support structured-data QA using Pandas/SQL.
- Implement agentic and self-correcting retrieval.
- Benchmark retrieval, generation and system performance.

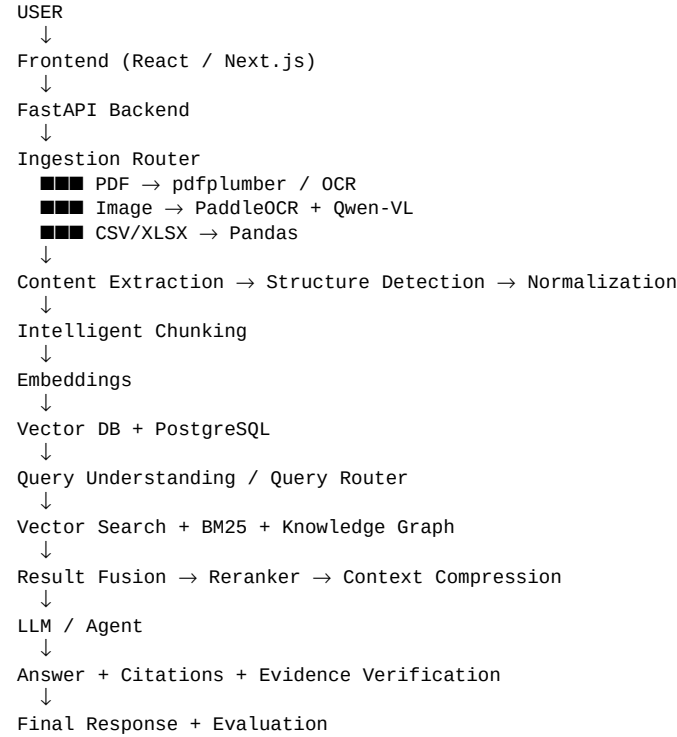
4. Target Users

- Students and researchers — papers, lecture notes, books and reports.
- Organizations and enterprises — policies, reports and internal documentation.
- Data analysts — CSV/Excel files and analytical questions.

5. Supported Input

Input	Processing
PDF	pdfplumber + OCR + VLM
Scanned PDF	OCR + VLM
Image	PaddleOCR + VLM
CSV	Pandas
XLSX	Pandas / OpenPyXL
DOCX	Document parser
TXT	Native parser
Markdown	Markdown parser
HTML / Web	HTML extraction

6. High-Level Architecture



7. Data Ingestion

7.1 PDF Pipeline

```
PDF
↓
PDF Type Detection
  ■■■ Text PDF → pdfplumber
  ■■■ Scanned PDF → OCR
↓
Text + Tables + Page Metadata
↓
Visual Elements → Qwen-VL
↓
Text / Tables / Images / Charts / Diagrams
```

7.2 Image Pipeline

```
Image
  ■■■ PaddleOCR → text extraction
  ■■■ Qwen-VL → visual understanding
    ■■■ charts
    ■■■ diagrams
    ■■■ screenshots
    ■■■ visual descriptions
```

7.3 CSV / Excel Pipeline

```
CSV / XLSX
↓
Pandas
↓
Schema detection
↓
Data profiling
↓
Structured representation
↓
columns / types / row count / statistics / missing values
```

8. Normalization

Create a common internal representation while preserving the original modality.

```
{
  "document_id": "doc_001",
  "page": 12,
  "content_type": "table",
  "text": "...",
  "table": {},
  "visual_description": null,
  "metadata": {}
}
```

Important metadata: document_id, chunk_id, page, section, source, content_type, created_at and user_id.

9. Intelligent Chunking

- Fixed-size chunking — baseline.
- Sentence-based chunking.
- Semantic chunking.
- Structure-aware chunking.
- Table-aware chunking.
- Multimodal chunking.

```
Document
  ■■■ Chapter
    ■■■ Section
      ■■■ Chunk
```

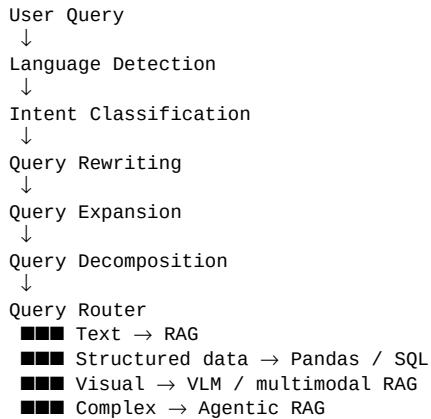
10. Embedding and Storage

Embeddings should be configurable through an `EmbeddingProvider` abstraction. Candidate models include BGE, E5 and Sentence Transformers or API-based embedding models.

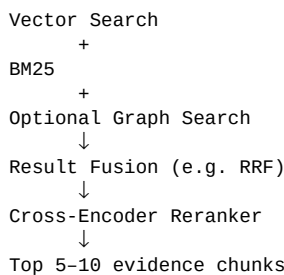
Vector storage: Qdrant, Pinecone or ChromaDB. Prefer one primary backend and expose a `VectorStore` abstraction for portability.

PostgreSQL: users, documents, chunks, conversations, messages, citations, evaluations and experiments.

11. Query Understanding and Routing



12. Hybrid Retrieval and Reranking

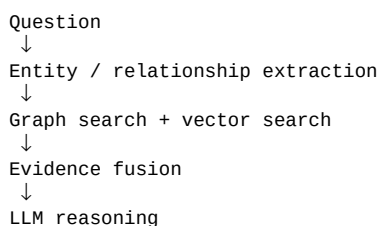


13. Context Compression

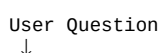
- Deduplicate retrieved content.
- Remove irrelevant information.
- Extract supporting evidence.
- Send the smallest useful evidence set to the LLM.

14. Knowledge Graph and Multi-Hop Reasoning

Extract entities, relationships, events, dates, organizations, people and products into a knowledge graph. Combine graph retrieval with vector retrieval for relationship-heavy and multi-hop questions.



15. Agentic / Self-Correcting RAG



```

Planner / Agent
↓
Search / Graph / SQL tools
↓
Evidence Evaluation
  ■■■ Enough → Generate
  ■■■ Not enough → Rewrite query → Search again
↓
Verified answer

```

16. Structured Data QA

Numerical and analytical questions over CSV/Excel should use Pandas or SQL for exact computation rather than relying only on embeddings.

```

Question
↓
Query Router
↓
Pandas / SQL
↓
Exact result
↓
LLM explanation

```

Mixed questions can combine SQL/Pandas results with document RAG before generation.

17. Answer Generation

- Use system instructions, user query, conversation context, retrieved evidence, structured results and source metadata.
- Enforce grounding: important claims must be supported by evidence.
- If evidence is insufficient, explicitly state that the available sources do not support a reliable answer.

18. Citation and Evidence Verification

```

Generated Answer
↓
Claim Extraction
↓
Evidence Matching
↓
Citation Verification
↓
Contradiction Detection
↓
Final Answer

```

Citation metadata should include document, page, section, chunk, source location and supporting text.

19. Contradiction Detection

When sources disagree, do not silently choose one. Report the conflict and apply an explicit source-authority policy where appropriate.

```

Source A → Revenue = ■50 Cr
Source B → Revenue = ■55 Cr
↓
Conflict detected
↓
Compare source authority / dates
↓
Explain which evidence is preferred and why

```

20. Conversation Memory

- Recent conversation memory.

- Conversation summaries.
- Relevant historical context.
- User-controlled view, edit, delete and disable controls.

21. Security

- JWT / OAuth authentication.
- Document-level authorization and metadata filtering before retrieval.
- File validation and upload limits.
- Prompt-injection defense: treat retrieved documents as untrusted data.
- SQL safety and input validation.
- Rate limiting and secrets management.

22. Evaluation Framework

Retrieval Metrics

- Recall@K
- Precision@K
- MRR
- NDCG

Generation Metrics

- Faithfulness
- Answer Relevance
- Context Relevance
- Citation Accuracy

System Metrics

- Latency
- Token usage
- Cost
- Throughput

23. Benchmarking and Experiments

Build a benchmark containing questions, expected answers and expected sources. Compare:

```

Vanilla RAG
vs
Hybrid RAG
vs
Hybrid + Reranker
vs
Multimodal RAG
vs
Agentic RAG

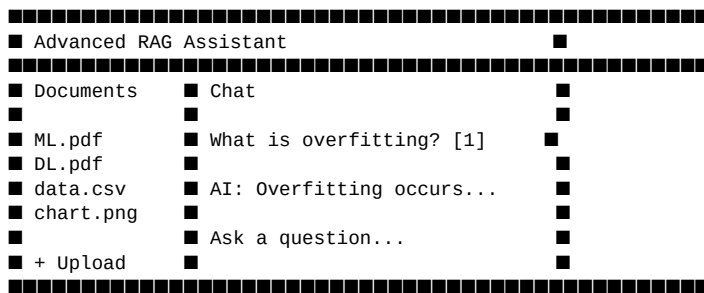
```

Example experiment dimensions: chunk size, overlap, embedding model, retrieval top-K, reranker, query rewriting and context compression.

24. Dashboard

- Retrieval metrics.
- Faithfulness and citation accuracy.
- Latency and token usage.
- Query traces from retrieval to final answer.
- Experiment comparison charts.
- Evaluation dataset results.

25. User Interface



26. Technology Stack

Layer	Recommended Technologies
Frontend	React / Next.js, Tailwind CSS
Backend	Python, FastAPI
Document Processing	pdfplumber, PaddleOCR, Pandas, OpenPyXL
Vision	Qwen-VL
RAG Framework	LangChain / LlamaIndex
Embeddings	BGE / E5 / Sentence Transformers
Vector DB	Qdrant / Pinecone / ChromaDB
Search	BM25, Hybrid Search
Database	PostgreSQL
Knowledge Graph	Neo4j
LLM	OpenAI / Gemini / Claude / Ollama
Evaluation	RAGAS + custom evaluation
Deployment	Docker / Docker Compose

27. Project Folder Structure

```

advanced-rag/
├── frontend/
├── backend/
│   ├── api/
│   ├── ingestion/
│   ├── processing/
│   ├── embeddings/
│   ├── retrieval/
│   ├── agent/
│   ├── generation/
│   ├── verification/
│   ├── evaluation/
│   └── database/

```

```
■ ■■■ main.py
■ ■■■ tests/
■ ■■■ evaluation/
■ ■■■ docker-compose.yml
■ ■■■ requirements.txt
■ ■■■ README.md
```

28. Development Phases

Phase 1 — Baseline: PDF → extraction → chunking → embeddings → vector DB → LLM

Phase 2 — Advanced Retrieval: Hybrid search → reranking → metadata filtering → query rewriting

Phase 3 — Multimodal: Images → OCR → Qwen-VL → tables → charts → diagrams

Phase 4 — Structured Data: CSV/Excel → Pandas/SQL → query router

Phase 5 — Knowledge Reasoning: Knowledge graph → multi-hop → temporal → cross-document reasoning

Phase 6 — Agentic RAG: Planner → decomposition → self-correction → tool use → adaptive retrieval

Phase 7 — Reliability: Citation verification → evidence checking → contradiction detection → confidence

Phase 8 — Evaluation: Benchmark dataset → automated evaluation → experiments → dashboard

29. Final System Capabilities

- Text QA — explain concepts from documents.
- Table QA — answer questions from extracted tables.
- Chart QA — understand trends and values in figures.
- Diagram QA — explain visual architectures and relationships.
- Multi-document comparison.
- Structured CSV/Excel analytics.
- Multi-hop and relationship questions.
- Temporal comparison across document versions.
- Conflict-aware answers.
- Agentic research-style questions involving search, calculation and synthesis.

30. Final Project Positioning

Recommended title: An Advanced Multimodal and Agentic Retrieval-Augmented Generation Framework for Evidence-Grounded Multi-Document Question Answering.

Core research question: How can hybrid retrieval, multimodal understanding, reranking, adaptive retrieval, knowledge graphs and agentic workflows improve the accuracy, faithfulness and efficiency of RAG-based question answering compared with vanilla RAG?

The project combines engineering and research: build the complete system, then experimentally measure whether each architectural improvement improves performance.

31. Final Architecture Progression

```
VANILLA RAG
↓
HYBRID RAG
(Vector + BM25)
↓
MULTIMODAL RAG
(PDF + Image + Table)
↓
```


ADVANCED RAG
(Reranking + Query Rewrite)



GRAPH RAG
(Knowledge Graph)



AGENTIC RAG
(Plan → Search → Verify → Answer)



VERIFIED RAG
(Evidence + Citations + Evaluation)